

StackScope, Question by Question

The full, comprehensive version — every important question about how StackScope actually works, answered in depth but in plain language, with the real code and fully worked examples.

Part 1 — The Big Picture: What Is This Thing, Really?

Q: What is StackScope, in one sentence, and why would anyone actually want it?

StackScope is a job-market intelligence tool. It automatically collects real, live job postings from several job sites, reads through each posting to figure out what skills it's asking for and what it pays, saves all of that into a database, and then answers three practical questions a developer might actually care about: “what skills are most in demand right now,” “which skills are trending up or down over time,” and “given the specific skills I already have, how well do I match the current market, and what salary should I expect.” The real problem it solves is that most people decide what to learn next based on gut feeling, old blog posts, or whatever's popular on social media — StackScope instead looks at what employers are *actually* posting, right now, and turns that into real numbers you can act on.

Q: If I had to describe the whole system as one continuous pipeline, what are the stages, in order?

There are four distinct stages, and it helps to think of them as a relay race where each stage hands something off to the next:

Stage	What happens	Where it lives
1. Scraping	Fetch raw job postings from external websites and APIs	scraper/*.py
2. Extraction	Pull structured facts (skills, salary, experience level) out of messy raw text	scraper/base.py (shared logic used during scraping)
3. Storage	Save every posting as one clean row in a database	database/models.py, database/db.py
4. Analysis	Answer real questions: matching, trends, salary prediction — served over HTTP	analysis/*.py, api/app.py

Notice that stages 1 and 2 actually happen together, in the same pass — as each posting is scraped, it's immediately run through skill/salary/experience extraction before ever touching the database. Nothing raw ever gets stored; only the already-cleaned-up version does.

Part 2 — Scraping: Where the Raw Data Actually Comes From

Q: What does “scraping” actually mean, mechanically? How is it different from just calling an API?

Some sources, like Adzuna, offer an official API — you send a request, and you get back clean, structured JSON, already organized into fields like title, company, and salary, built specifically for other programs to consume. Other sites don't offer that; all you get is the same HTML page a human visitor's browser would render. Scraping means downloading that raw HTML — which is really just a long string of text full of tags like `<div>`, `<span>`, and `<a>` describing layout and styling — and then picking out the specific pieces of text you actually care about from inside all that markup, using a library called BeautifulSoup that understands HTML structure well enough to let you say things like “find me the text inside every `<div class="job-item">`.”

Q: Why does every scraper share one common base — what problem is BaseScraper actually solving?

*Picture a recipe template used by every restaurant in a chain: “heat the pan, add the base sauce, plate it a certain way” — steps every location does identically. The only thing that changes location to location is the one specific dish being cooked. BaseScraper is exactly that shared template: fetching a page politely, figuring out what skills are mentioned, guessing the experience level, and cleaning up salary text are all done identically everywhere. The only thing each specific site's scraper subclass has to supply is its own answer to “how do I read *this* particular site's page layout.”*

This is a real object-oriented programming pattern called an abstract base class. In Python, BaseScraper is defined using Python's ABC (Abstract Base Class) module, and it marks one method, `scrape()`, as abstract — meaning every subclass (PNetScraper, and the ones actually wired in like AdzunaScraper) is *required* to provide its own version of `scrape()`, or Python will refuse to let you create that subclass at all. Here's the shared foundation every scraper inherits automatically:

```

class BaseScraper(ABC):

    def __init__(self, source: dict):
        self.source = source
        self.name = source["name"]
        self.url = source["url"]
        self.country = source["country"]
        self.headers = {"User-Agent": "Mozilla/5.0 ..."}

    def fetch(self, url: str) -> BeautifulSoup | None:
        try:
            response = requests.get(url, headers=self.headers, timeout=REQUEST_TIMEOUT)
            response.raise_for_status()
            time.sleep(SCRAPE_DELAY)
            return BeautifulSoup(response.text, "html.parser")
        except requests.exceptions.Timeout:
            return None
        # ...other specific error types handled the same way, each just logged and skipped

    @abstractmethod
    def scrape(self) -> list[dict]:
        pass

```

Every one of the specific fetch errors — timeout, HTTP error, generic request failure — is caught separately and turned into a warning log plus a clean None return value, rather than crashing the entire scrape run over one bad page. That matters a lot in practice: if scraping stopped completely the moment any single page failed to load, one flaky page on one site could prevent every other posting on every other site from ever being collected that run.

Q: Walk me through, in full detail, what happens to one single real job posting — from the raw HTML on the page all the way to a saved row in the database.

Let's trace the real PNetScraper end to end. First, scrape() loops through result pages one at a time (up to a configured MAX_PAGES), calling self.fetch() on each page's URL — which, as shown above, downloads the page, waits politely, and hands back a parsed BeautifulSoup object. It then searches that page for every <div class="job-item"> block — each one representing exactly one job posting — and hands each block to _parse_listing():

```
def _parse_listing(self, listing) -> dict | None:
    title_tag = listing.find("h2", class_="job-title")
    company_tag = listing.find("span", class_="company-name")
    salary_tag = listing.find("span", class_="salary")
    description_tag = listing.find("div", class_="job-description")

    title = title_tag.get_text(strip=True) if title_tag else None
    if not title:
        return None    # a listing with no title is treated as unusable, skipped entirely

    salary_raw = salary_tag.get_text(strip=True) if salary_tag else None
    description = description_tag.get_text(strip=True) if description_tag else ""

    salary_min, salary_max = self.parse_salary(salary_raw)
    skills = self.extract_skills(title + " " + description)
    experience_level = self.detect_experience_level(title + " " + description)

    return { "title": title, "salary_min": salary_min, "salary_max": salary_max,
            "skills": skills, "experience_level": experience_level, ... }
```

Notice the defensive pattern repeated for every single field: “grab the tag if it exists, otherwise fall back to None or an empty string.” Real HTML pages are inconsistent — some listings might be missing a salary tag entirely, some might have a slightly different layout — and this pattern means one missing field doesn't crash the whole listing, it just leaves that one field empty. Only a genuinely missing title causes the whole listing to be thrown away, since a job posting with no title at all isn't really usable data. Once this dictionary comes back, run.py loops over every returned job and calls `save_job()` on each one (covered fully in Part 3), which is the actual moment the posting becomes a permanent database row.

Q: Why does the scraper deliberately wait (`time.sleep(SCRAPE_DELAY)`) between every single request, instead of just firing them all off as fast as possible?

Picture the difference between someone ringing a shop's doorbell once every few seconds, politely, versus someone hammering on the door nonstop as fast as their arm can move. The second one looks exactly like someone trying to break in, and any reasonable shopkeeper (or website) is going to react to it as a threat, not a customer.

Sending requests as fast as code physically can is functionally indistinguishable from a denial-of-service attack from the website's point of view, and most sites will detect that pattern and block the offending IP address — which would break the scraper for good, not just slow it down once. `SCRAPE_DELAY` makes every single request wait a configured amount of time before the next one goes out, which keeps request volume looking like a normal, occasional visitor. It's slower, deliberately, but it's the only sustainable way to keep scraping a site you don't own without eventually losing access to it entirely — and it's also just the ethically correct way to interact with someone else's server.

Q: There are scraper files for PNet, Careers24, and Indeed sitting in the codebase — but config.py's SOURCES dictionary, and run.py's SCRAPER_MAP, only reference Adzuna, Remotevive, We Work Remotely, Jobspresso, and JobLeads. What's actually going on there, and how should I explain it if asked?

This is a completely real and, importantly, a completely honest thing to know about your own project rather than be caught off guard by: PNetScraper, a Careers24 scraper, and an Indeed scraper are all fully working pieces of code — they'd run correctly if invoked — but none of them are ever actually invoked, because SOURCES (which run.py reads to decide what to scrape) and SCRAPER_MAP (which maps a source's key to its scraper class) simply don't mention them. They're either an earlier direction that got replaced by the currently-enabled five sources, or scrapers built ahead of actually needing them. The right way to describe this in an interview is exactly that directly: “these three exist and work, but aren't currently wired into the active source list — I'd either finish integrating them or remove them.” That's a stronger, more self-aware answer than hoping nobody notices, and it's also a recognizable pattern across your other projects worth naming out loud: unused Qdrant code and unused Alembic migrations in DocuZen, and unused spaCy here too (Part 4) — you have a habit of building more than what ends up wired in, and being able to name that pattern yourself is a mark of self-awareness, not a weakness.

Part 3 — Storage: SQLite, PostgreSQL, and Avoiding Duplicate Jobs

Q: What does one saved job posting actually look like in the database, field by field?

Every posting becomes one row in a jobs table with these columns: id (an auto-incrementing number identifying the row), title, company, location, salary_raw (the original, messy salary text, kept for reference), salary_min and salary_max (the cleaned-up numeric values), description, skills (a comma-separated string like “python, django, docker”), experience_level, source (which site it came from), country, url (kept unique — this is important, see below), date_posted, and date_scraped.

Q: Why does the code support both SQLite and PostgreSQL — why not just pick one?

This comes straight from a real, honest lesson described in your own README: the project originally used SQLite, a simple file-based database that needs no separate server — great for developing locally on your own laptop. But once deployed to Render (a hosting platform), Render's free tier uses an ephemeral filesystem, meaning any file written to disk — including a SQLite database file — gets wiped out every time the service restarts or goes cold. That silently destroyed the entire scraped dataset on every restart. The fix was migrating to PostgreSQL (hosted separately, on Supabase), which lives outside the app's own filesystem entirely and survives restarts. The code still supports both, toggled automatically based on whether a DATABASE_URL environment variable is set:

```

DATABASE_URL = os.environ.get("DATABASE_URL", "")
BACKEND = "postgres" if DATABASE_URL else "sqlite"

def get_connection():
    if BACKEND == "postgres":
        return psycopg2.connect(DATABASE_URL)
    conn = sqlite3.connect(DATABASE_PATH)
    conn.row_factory = sqlite3.Row
    return conn

```

This means local development can stay simple (just a file, no separate database server to install and run), while the real deployed version automatically switches to the durable option, entirely based on whether that one environment variable happens to be set — no code change required to switch between them.

Q: The scraper runs repeatedly over time — what stops the exact same job posting from being saved again and again every time the scraper re-runs and encounters it a second time?

Imagine a guest list at the door of an event, where every name has to be unique. If someone tries to sign in under a name that's already on the list, the door staff just quietly wave them through without adding a duplicate entry — the list doesn't grow, and nobody has to manually check for repeats themselves.

The url column is declared UNIQUE in the table schema, and the actual insert statement uses PostgreSQL's ON CONFLICT (url) DO NOTHING — meaning “try to insert this row, but if a row with this exact URL already exists, silently skip it instead of raising an error.” Since a real job posting's URL is a genuinely stable, unique identifier for that specific posting, this is a clean, database-level way to guarantee the scraper can be safely re-run as often as you like — daily, hourly, whatever — without ever quietly duplicating the same posting into the dataset twice and skewing every count and percentage calculated from it.

```

cur.execute("""
    INSERT INTO jobs (title, company, ..., url, date_posted, date_scraped)
    VALUES (%(title)s, %(company)s, ..., %(url)s, %(date_posted)s, %(date_scraped)s)
    ON CONFLICT (url) DO NOTHING
    RETURNING id
""", {**job, "date_scraped": datetime.now().isoformat()})
return cur.fetchone() is not None    # True only if a NEW row was actually inserted

```

Part 4 — Turning Messy Text Into Structured Facts

Q: How, precisely, does StackScope know a job posting is asking for “Python” or “Docker”? Is any of this actually AI or natural language understanding?

No — and it's important to be precise about this rather than let anyone assume otherwise. This is keyword matching against a fixed, hand-written list, full stop. `config.py` defines a Python list called `SKILLS` containing roughly 60 specific strings — “python”, “docker”, “react”, “kubernetes”, and so on, grouped loosely by category (languages, frontend, backend, data/ML, databases). `base.py` pre-compiles one regular expression per skill, once, when the module is first loaded — not freshly for every single job posting, which would be wasteful:

```
_SKILL_PATTERNS = [
    (skill, re.compile(r'\b' + re.escape(skill) + r'\b', re.IGNORECASE))
    for skill in SKILLS
]

def extract_skills(self, text: str) -> str:
    found = [skill for skill, pattern in _SKILL_PATTERNS if pattern.search(text)]
    return ", ".join(found)
```

`extract_skills()` simply checks, for every one of those ~60 pre-built patterns, whether it appears anywhere in the posting's combined title-plus-description text, and joins every skill that matched into one comma-separated string like “python, docker, kubernetes”. That entire string is what gets saved into the skills column, and it's also exactly the same format later split back apart by every analysis function (`skills.py`, `matcher.py`, `trends.py` all call `.split(",")` on it).

Q: What is a “regular expression” (regex), in plain terms, and why does this one specifically need a word boundary (`\b`)?

A regex is just a precise pattern for searching text — instead of asking “does this exact word appear,” you can ask more flexible questions like “does this appear anywhere, in any capitalization, as long as it's a whole standalone word and not just a fragment buried inside a bigger word.” Without that last condition, searching for the letter “r” would treat the word “bar”, “career”, “server”, and “area” as all matching — because they all technically contain the letter r somewhere inside them.

`re.escape(skill)` safely handles skills containing special regex characters — “c++” and “c#” contain characters (+ and #) that would otherwise be misinterpreted as regex instructions rather than literal characters to search for. `\b` on both sides of the pattern means “only count this as a match if there's a word boundary (like a space, punctuation, or the start/end of the text) immediately on both sides” — in other words, match the whole word “python”, never just the letters “python” sitting in the middle of some longer, unrelated word. `re.IGNORECASE` means the check doesn't care whether the posting wrote “Python”, “PYTHON”, or “python” — all count as a match.

Q: You mentioned there was a real bug here — what actually happened, and how does it show word boundaries alone aren't always enough?

`config.py`'s `SKILLS` list has a comment directly documenting this: the single letter “r” (for the R programming language) used to be in the list, and it caused false-positive matches constantly — even

with word boundaries protecting against it appearing mid-word, a single standalone letter is still extremely likely to appear as its own “word” by pure coincidence somewhere in a long job description (initials, abbreviations, stray formatting characters). The fix wasn't a cleverer regex — it was simply removing “r” from the tracked skills list entirely, accepting that a single-letter skill name isn't reliably detectable this way at all. This is a great, concrete debugging story for an interview: you shipped something, it produced genuinely wrong output on real data, you diagnosed exactly why, and you fixed it with a deliberately simple, honest solution rather than over-engineering a workaround.

Q: spaCy is listed in requirements.txt, and the README's tech-stack table even mentions it — so where is it actually used?

Nowhere. I searched the entire codebase for the word “spacy” and it does not appear anywhere outside of requirements.txt and the dependency's own installed files — not imported, not called, not referenced once in any of your own code. The same is true of textblob, also listed. The honest, precise answer, if this comes up: “skill extraction is regex-based keyword matching against a fixed list, not NLP — spaCy is a dependency left over from an earlier direction I didn't end up needing, and I should remove it.” That's a materially stronger answer than letting an interviewer assume real natural-language-model-based extraction is happening when it isn't — precision about your own system, including its limitations, reads as far more credible than vague buzzword use.

Q: How does experience level (junior / mid / senior) get decided?

The same keyword-matching philosophy, applied to a different, smaller set of words:

```
def detect_experience_level(self, text: str) -> str:
    text_lower = text.lower()
    if any(w in text_lower for w in ["senior", "lead", "principal", "staff"]):
        return "senior"
    if any(w in text_lower for w in ["mid", "intermediate", "3+ years", "4+ years"]):
        return "mid"
    if any(w in text_lower for w in ["junior", "graduate", "entry", "intern", "0-2", "1-2"]):
        return "junior"
    return "unspecified"
```

Notice the order matters: it checks for senior-indicating words first, then mid, then junior — so a posting mentioning both “senior” and “3+ years” would be classified senior, because that check happens first and returns immediately. Anything matching none of these falls through to “unspecified” rather than being forced into a guess — an honest way of saying “this data point isn't reliable enough to classify.”

Part 5 — The Matching Engine: TF-IDF and Cosine Similarity, Fully Explained

Q: What is TF-IDF, really — not just the acronym, but what it's actually computing?

TF-IDF stands for Term Frequency times Inverse Document Frequency, and it's built from two separate ideas multiplied together.

Term Frequency (TF)

— simply, how often a specific term appears within one specific document. In StackScope's case, each “document” is a job's comma-separated skill string, and each skill in that list typically appears once, so TF here is mostly just “is this skill present in this posting's skill list at all.”

Inverse Document Frequency (IDF)

— this is the more interesting half. It measures how rare a term is *across the entire collection* of documents, computed roughly as $\log(\text{total number of documents} \div \text{number of documents containing this term})$. A skill that appears in almost every posting produces a small IDF value (because the denominator, the count of documents containing it, is large, making the whole fraction close to 1, and $\log$ of a number close to 1 is close to 0). A rare skill, appearing in only a few postings, produces a much larger IDF value.

Multiplying $\text{TF} \times \text{IDF}$ means: a skill only contributes a large final number to a document's vector if it's both actually present in that document AND rare across the whole collection. A common skill present in a posting contributes only a small number; a rare skill present in a posting contributes a large number. That's the entire mechanism — no more complex than that.

Q: Can you show the actual arithmetic, with a small made-up example?

Take four job postings and their skill lists:

Posting	Skills listed
Job A	python, django, postgresql
Job B	python, react, javascript
Job C	python, django, docker
Job D	rust, systems programming

“python” appears in 3 of these 4 documents. Its IDF is roughly $\log(4/3) \approx 0.29$ — a small number, because it's common. “django” appears in 2 of 4 documents: $\log(4/2) = \log(2) \approx 0.69$ — noticeably larger, because it's rarer. “rust” appears in only 1 of 4: $\log(4/1) = \log(4) \approx 1.39$ — the largest weight of all, because it's the rarest. This is the exact mathematical reason “django” counts for more than “python” when comparing a user's profile against these postings — it isn't a vague intuition, it's a real, computable number baked directly into each document's vector before any comparison even happens.

Q: Now, given a user who lists “python, django” as their own skills, how does the actual code compare that against all four postings, and what does it return?

```
def match_jobs(user_skills: list[str], top_n: int = 10, ...):
    jobs = get_all_jobs()
    job_skill_strings = [job["skills"] for job in jobs if job["skills"]]
    user_skill_string = ", ".join(user_skills)

    all_documents = job_skill_strings + [user_skill_string]

    vectorizer = TfidfVectorizer()
    tfidf_matrix = vectorizer.fit_transform(all_documents)

    user_vector = tfidf_matrix[-1]      # the user's own vector – always the LAST row
    job_vectors = tfidf_matrix[:-1]    # every job's vector – everything BEFORE the
last row

    similarities = cosine_similarity(user_vector, job_vectors)[0]
    top_indices = np.argsort(similarities)[::-1][:top_n]  # highest similarity first
```

The cleverest line here is `all_documents = job_skill_strings + [user_skill_string]` — the user's own skill list is temporarily added to the *same* collection of documents as every job posting, so that `TfidfVectorizer` computes IDF weights across all of them together, consistently, in one single call. Because the user's string was appended last, `tfidf_matrix[-1]` (the last row) is always guaranteed to be the user's own vector, and everything before it (`tfidf_matrix[:-1]`) is every job's vector. `cosine_similarity` then measures the angle between the user's vector and every single job vector at once, returning one similarity score per job, and `np.argsort(...)[::-1]` sorts those scores from highest to lowest so the best matches come first.

For our four-job example, the user's vector would have a real, nonzero weight only on “python” and “django” (the two skills they actually listed) — everything else stays at zero. Job A's vector has weight on “python”, “django”, and “postgresql”; Job C's has weight on “python”, “django”, and “docker”. Both A and C share the user's two weighted terms, including the more heavily-weighted “django”, so their cosine similarity to the user comes out highest. Job B shares only “python” (the lower-weighted, more common term), so it scores lower. Job D shares nothing at all with the user, so its similarity is essentially zero.

Q: What exactly is `get_gap_score()` doing, and how is it different from `match_jobs()`?

Where `match_jobs()` ranks individual postings against a user, `get_gap_score()` answers a slightly different, more personal question: “out of everything the market is currently asking for, how much of it do I personally already have?” It collects every skill mentioned across every posting, counts them with Python's `Counter`, and takes the top 20 most frequently requested skills market-wide. It then checks how many of those top 20 the user's own list already covers, and turns that into a single percentage:

```
have = [s for s in top_skills if s in user_lower]
missing = [s for s in top_skills if s not in user_lower]
score = round((len(have) / len(top_skills)) * 100, 1)
```

If a user covers 14 of the top 20 most in-demand skills, their score is 70.0 — and the missing 6 become an actual, concrete, personalized “here's specifically what to learn next” list, rather than a generic suggestion.

Q: Is any of this — TF-IDF, cosine similarity, the gap score — actually machine learning?

No, and this is one of the most important distinctions to be able to draw cleanly, for either a software-engineering or an AI-engineering interview. Machine learning means a model with parameters that get **learned** from training data, and which can then generalize to new, unseen inputs based on patterns found during that training. TF-IDF and cosine similarity involve no training step at all — every time `match_jobs()` runs, it recomputes everything completely from scratch, from whatever data currently happens to be in the database. Nothing is remembered or learned between calls. This is applied statistics and linear algebra — genuinely useful, genuinely worth understanding deeply, but categorically not machine learning. There is exactly one real trained machine learning model anywhere in this project, covered next.

Part 6 — The One Real Machine Learning Model: Predicting Salary

Q: What is linear regression, in the simplest possible terms, before looking at the code?

Imagine plotting points on a graph — each point is one job posting, positioned by its skills and experience level on one axis and its actual salary on the other. Linear regression is the process of finding the single straight line (or, with more than one input, a flat plane) that sits as close as possible to all of those points at once. Once you have that line, you can hand it a brand new point's position — a skill set it hasn't seen paired with a salary before — and read off roughly where that line predicts the salary should sit.

Mathematically, a simple version looks like $\text{salary} = b_0 + b_1 \times (\text{has_python}) + b_2 \times (\text{has_django}) + \dots + b_n \times (\text{experience_level})$, where each b is a coefficient the model learns from the training data — essentially, “how many extra rand or dollars does having this specific skill tend to add to the predicted salary, based on the real, historical postings we've actually seen.”

Q: Skills are words, like “python” and “react” — how does a model that only understands numbers use that as an input at all?

Picture a spreadsheet with one column for every possible skill that's ever shown up across all the scraped postings. For any one specific posting, every column gets a 1 if that posting asked for that skill, and a 0 if it didn't. A posting listing "python, react" becomes one row that's mostly zeros, with a single 1 under the python column and a single 1 under the react column, and 0 under every other skill column that exists.

This is exactly what scikit-learn's `MultiLabelBinarizer` does, and it's a standard technique called one-hot encoding. Here's a small worked table showing what it produces for three postings, if the model only knew about four possible skills total:

Posting	python	django	react	docker
Job A (python, django)	1	1	0	0
Job B (python, react)	1	0	1	0
Job C (python, django, docker)	1	1	0	1

Each of those four columns becomes one of the "x" inputs to the regression, alongside one more numeric column representing experience level (junior=0, mid=1, senior=2). The model then learns one coefficient per column — in effect, learning "how much does each individual skill, and each step up in seniority, tend to add to salary, based on everything we've actually scraped so far."

Q: Show me the actual training and prediction code, and walk through it.

```

def predict_salary(skills: list[str], experience_level: str) -> dict | None:
    jobs = get_all_jobs()
    labelled = [j for j in jobs if j["salary_min"] and j["skills"] and j["experience_level"]]

    if len(labelled) < 10:
        return None    # refuses to guess with too little real data

    mlb = MultiLabelBinarizer()
    skill_lists = [[s.strip() for s in job["skills"].split(",")] for job in labelled]
    X_skills = mlb.fit_transform(skill_lists)    # one-hot encode every
posting's skills
    X_experience = np.array([EXPERIENCE_MAP.get(job["experience_level"], 1)
                           for job in labelled]).reshape(-1, 1)
    X = np.hstack([X_skills, X_experience])    # combine into one input
table
    y = np.array([job["salary_min"] for job in labelled])    # the real, known answers

    model = LinearRegression()
    model.fit(X, y)    # THIS is the actual tra
ining step

    input_vector = np.hstack([mlb.transform([skills]),
                             [[EXPERIENCE_MAP.get(experience_level, 1)]]])
    prediction = model.predict(input_vector)[0]

    return {
        "predicted_min": round(max(prediction * 0.85, 0), 2),
        "predicted_max": round(prediction * 1.15, 2),
        "confidence": "low" if len(labelled) < 50 else "medium" if len(labelled) < 20
0 else "high",
    }

```

`model.fit(X, y)` is the one genuine moment of “learning” anywhere in this whole project — scikit-learn is finding the coefficients that make the model's predictions fit the real, historical `salary_min` values as closely as mathematically possible, across every labelled posting available right now. Every time this function runs, it retrains completely from scratch on whatever data currently exists — it isn't a saved, reused model file, it's trained fresh, live, per request.

Q: Can you walk through one full prediction, start to finish, with made-up but concrete numbers?

Step	What happens
Input	<code>skills = ["python", "django"]</code> , <code>experience_level = "mid"</code>
One-hot encode	<code>python=1</code> , <code>django=1</code> , every other tracked skill=0
Experience number	<code>EXPERIENCE_MAP["mid"] = 1</code>
Combined input row	<code>[1, 1, 0, 0, ..., 1]</code> ← last value is the experience number
Model's raw output	<code>model.predict(...)</code> returns a single number, say 42,000

Step	What happens
Final range returned	$\text{predicted_min} = 42,000 \times 0.85 = 35,700$ $\text{predicted_max} = 42,000 \times 1.15 = 48,300$

The 0.85 / 1.15 multipliers are a simple, deliberate way of turning one raw predicted number into an honest range rather than a falsely precise single figure — acknowledging that a linear regression's single output is a rough central estimate, not a guaranteed exact answer.

Q: What does the “confidence: low / medium / high” label actually mean — is it a real statistical accuracy measure?

No, and this is an important, honest limitation to be ready to name yourself. It's purely a count of how much labelled training data existed at the moment of prediction: fewer than 50 usable postings is labelled “low” confidence, under 200 is “medium,” 200 or more is “high.” There is no R^2 score, no error measurement, no validation against held-out test data anywhere in this function — confidence here is a proxy for “how much data did the model get to learn from,” not a real measurement of “how accurate are its predictions actually turning out to be.” A strong, honest answer if asked “how do you know this model is any good”: “I don't have a real accuracy metric yet — that's a genuine next step, evaluating predictions against a held-out set of real salaries rather than just gating on sample size.”

Part 7 — Detecting Whether a Skill Is Trending

Q: How does the system turn raw scraped postings into a week-by-week trend?

```
def get_weekly_skill_counts() -> dict:
    jobs = get_all_jobs(limit=5000)
    weekly = defaultdict(lambda: defaultdict(int))

    for job in jobs:
        date = datetime.fromisoformat(job["date_scraped"])
        week = date.strftime("%Y-W%W")      # e.g. "2026-W28"
        for skill in job["skills"].split(", "):
            weekly[week][skill] += 1

    return {week: dict(counts) for week, counts in sorted(weekly.items())}
```

Every posting gets bucketed into the calendar week it was scraped (year + week number, like “2026-W28”), and for each of those weekly buckets, every skill mentioned in that week's postings gets its count incremented by one. The result is a dictionary of dictionaries: for each week, how many times was each skill mentioned that week.

Q: Once you have weekly counts, how does the code decide a skill is genuinely “rising” rather than just having one noisy good week?

*Picture plotting a skill's weekly count as dots on a graph, one dot per week, left to right. Fitting a straight line through those dots and checking whether that line tilts upward or downward gives you a single, stable sense of the overall direction — smoothing over the fact that any *individual* week might randomly spike or dip for reasons that have nothing to do with a real underlying trend.*

`np.polyfit(x, y, 1)` does exactly that — it fits the best possible straight line through a set of points and returns that line's slope (how steeply it rises or falls) and intercept. Here's a tiny, concrete example: say “rust” was mentioned in 2 postings in week 1, 3 in week 2, 5 in week 3, and 8 in week 4. Fitting a line through the points (1,2), (2,3), (3,5), (4,8) produces a clearly positive slope — even though the week-to-week jumps aren't perfectly even (+1, then +2, then +3), the overall line still tilts upward across all four weeks combined, which is exactly the kind of single, stable number you want instead of reacting to any one individual jump.

```
def _calculate_direction(counts: list[int]) -> tuple[str, float]:
    x = np.array(range(len(counts)), dtype=float)
    y = np.array(counts, dtype=float)
    slope = np.polyfit(x, y, 1)[0]

    first_half = np.mean(y[:len(y)//2])
    second_half = np.mean(y[len(y)//2:])
    change_percent = ((second_half - first_half) / first_half) * 100

    if slope > 0.5 and change_percent > 10:
        direction = "rising"
    elif slope < -0.5 and change_percent < -10:
        direction = "declining"
    else:
        direction = "stable"
    return direction, change_percent
```

Notice it requires *two separate signals to agree* before committing to a label: the slope has to be meaningfully positive (greater than 0.5, not just barely above zero), AND the second half of the tracked weeks has to average at least 10% higher than the first half. Either check alone could be fooled by one unusual week; requiring both to agree at once is a simple but genuinely effective way of filtering out noise before calling something a real trend.

Part 8 — Cleaning Up Messy Real-World Salary Text

Q: Why is salary data specifically so much harder to work with than, say, a job title?

Because it's written by hundreds of different real people across different countries, with zero shared format. The exact same actual salary could be written as “R45,000 per month”, “R540k per annum”, “\$3,750/month (USD)”, or “£25/hour” — four wildly different strings that might represent very similar real income. Nothing about that text is structured or consistent, so before any comparison across

postings is even meaningful, it all has to be normalized down to one shared, consistent unit.

Q: Walk through the actual normalization logic and a few realistic examples.

```
def normalise_salary(salary_raw: str, period: str = "monthly"):
    text = salary_raw.lower().replace(",", "").replace(" ", "")
    is_annual = any(w in text for w in ["year", "annum", "annual", "pa", "yr"])
    is_hourly = "hour" in text or "/hr" in text

    numbers = re.findall(r"\d+(?:\.\d+)?k?", text)
    cleaned = []
    for n in numbers:
        if n.endswith("k"):
            cleaned.append(float(n[:-1]) * 1000)    # "45k" -> 45000
        else:
            val = float(n)
            if val > 500:                            # filters out stray small numbers,
                cleaned.append(val)                  e.g. "5 days/week"

    salary_min = min(cleaned)
    if is_annual:
        salary_min = salary_min / 12
    elif is_hourly:
        salary_min = salary_min * 160                # assumes ~160 working hours per month
    return round(salary_min, 2), ...
```

Raw text	Detected as	Normalized to monthly
"R45,000 per month"	monthly, no conversion needed	R45,000
"R540k per annum"	annual, "k" shorthand	$540,000 \div 12 = \text{R}45,000$
"\$85k/yr"	annual, USD, "k" shorthand	$85,000 \div 12 \approx \$7,083$
"£25/hour"	hourly, GBP	$25 \times 160 \approx \text{£}4,000$

The `val > 500` check is a small but deliberate safeguard: real job postings sometimes contain small, unrelated numbers near the salary text (like "5 days/week" or a listing ID), and this filters those out before they could be mistaken for part of the actual salary figure.

Q: Is there anything genuinely messy about how this specific piece is implemented — something worth flagging before an interviewer finds it first?

Yes, and it's worth naming proactively. This exact normalization logic — detect the period, strip commas, expand "k" notation, convert to a monthly baseline — exists in two separate places: `normalise_salary()` here in `analysis/salary.py`, and a near-identical `parse_salary()` method inside `scraper/base.py`. The same real problem got solved twice, independently, in two different files, instead of being written once as a shared utility both could call. A clean, self-aware answer if asked "what would you refactor first": "consolidate the duplicated salary-parsing logic in `scraper/base.py` and

analysis/salary.py into one shared function, so a future fix only has to happen in one place.”

Part 9 — Serving It All Through an API

Q: What is Flask actually doing here, and what happens end to end for one real request?

Flask is a lightweight Python web framework — its whole job is mapping an incoming HTTP request (a specific URL plus a method like GET or POST) to a specific Python function, and turning that function's return value into an HTTP response the caller can use. Trace one real example: a request arrives at GET /api/skills/frequency?limit=10. Flask's `@app.route("/api/skills/frequency")` decorator means this exact path is wired directly to the `skill_frequency()` function. That function reads the `limit` query parameter (defaulting to 20 if not provided) and calls `get_skill_frequency(limit=10)` from `analysis/skills.py` — which fetches every job, splits every posting's comma-separated skills string apart, counts every individual skill across all postings using Python's `Counter`, and returns the 10 most common as a list of dictionaries. Flask's `jsonify()` then converts that Python list straight into a proper JSON HTTP response.

Q: What is CORS, and why does a project like this specifically need it configured?

Imagine a strict rule at a building's front desk: “only let visitors in who arrived from this specific, pre-approved list of addresses — turn everyone else away automatically, no exceptions, no explanations needed.” Web browsers enforce an equivalent rule by default for any webpage trying to call an API hosted on a different domain than the page itself.

This default browser restriction is called the Same-Origin Policy, and CORS (Cross-Origin Resource Sharing) is the specific mechanism a server uses to explicitly say “this other origin is allowed to call me.” Since StackScope's frontend (plain static HTML/JS) and its backend (this Flask API, typically deployed separately on Render) live on two different domains, the browser would silently block every API call from the frontend by default unless the backend explicitly grants permission. `flask_cors.CORS(app, resources={r"/api/*": {"origins": ...}})` is exactly that explicit grant — without it, the frontend would appear completely broken, with every request quietly failing in the browser before it even reached the server.

Part 10 — Honest Limitations, All in One Place

Limitation	What's really going on, and why it matters
spaCy and textblob are unused dependencies	Both are listed in <code>requirements.txt</code> and never imported anywhere in your own code. Same recurring pattern as unused Qdrant/Alembic in DocuZen — worth naming as a pattern you're now aware of.

Limitation	What's really going on, and why it matters
Three scraper files are dead code	pnet.py, a Careers24 scraper, and an Indeed scraper all work but aren't wired into SOURCES or SCRAPER_MAP, so they never actually run.
Salary parsing logic is duplicated	Nearly identical normalization exists separately in scraper/base.py and analysis/salary.py — the same problem solved twice instead of shared.
"Confidence" isn't a real accuracy measure	It's a sample-size threshold only (low/medium/high) — there's no R^2 or held-out validation checking whether predictions are actually correct.
Keyword matching can't understand context or negation	A posting saying "no Python experience required" still gets flagged as requiring Python — presence of a word is detected, not its actual meaning in context.
Skill extraction is limited to a fixed, hand-maintained list	Any skill not already typed into config.py's SKILLS list can never be detected, no matter how many postings mention it.

Part 11 — Interview Cheat Sheet: Ready-Made Answers

Q: "Walk me through what StackScope does and how it's built."

"It scrapes live job postings from several job boards and APIs, using a shared base scraper class so every site follows the same fetch-politely-then-extract pattern. Each posting gets its skills extracted via regex keyword matching against a fixed list, its salary normalized into a consistent monthly figure regardless of how it was originally written, and its experience level guessed from keywords — all before being saved into Postgres with a unique constraint on the URL so re-running the scraper never creates duplicates. On top of that stored data, a Flask API serves skill-frequency counts, trend detection using linear regression on weekly counts, TF-IDF and cosine-similarity matching between a user's skills and the market, and a genuinely trained linear regression model that predicts a salary range for a given skill set."

Q: "What's the difference between the matching engine and the salary predictor, conceptually?"

"The matching engine — TF-IDF plus cosine similarity — is applied statistics: it's recomputed completely fresh on every single request, with nothing learned or stored between calls. The salary predictor is genuine machine learning: it actually trains a linear regression model on real, one-hot-encoded historical data using `model.fit()`, producing learned coefficients that generalize to skill combinations it hasn't seen paired with a salary before. Being able to draw that line precisely matters more than calling everything 'AI.'"

Q: "What's the weakest part of this project, and what would you fix first?"

"A few honest ones, in order of how much I'd prioritize them: keyword-based skill extraction can't tell a required skill apart from one a posting explicitly says it doesn't need. The salary prediction's

'confidence' label is just a sample-size threshold, not a real accuracy metric — I'd add proper validation against held-out data. My salary-normalization logic is duplicated across two files instead of one shared function. And I have a few unused dependencies and dead scraper files I'd either wire in properly or remove."

Q: "What would you build next if you kept developing this?"

"Move skill extraction toward something context-aware rather than pure keyword presence, so it can handle negation and required-vs-nice-to-have distinctions. Add a real evaluation step for the salary model — training on part of the data and measuring prediction error on the rest, rather than just gating confidence on how much data exists. And consolidate the duplicated salary-parsing code into one shared function the scraper and the analysis layer both call."